Practical Session 1: Introduction to Java & OOP Fundamentals

Student-Centered, Self-Paced Lab

✓ Learning Outcomes Covered

By the end of this practical session, you will be able to:

1. **Explain** the differences between procedural and object-oriented programming paradigms using a simple example.
2. **Classify** Java syntax elements (class, main method, statements, comments) in a given code snippet.
3. **Demonstrate** the ability to compile and run a Java program independently.
4. **Compare** the roles of JVM, JRE, and JDK in executing Java programs.

Time Required: 90–120 minutes
Instructor Intervention: None – you will figure it out yourself!

✓ Before You Begin

Ensure you have completed **Lab 1** (VS Code, Git, GitHub, JDK installed). You will need:

- VS Code open
- A folder called java-course with a subfolder week1
- Git configured and connected to GitHub

✓ Your Roadmap



TASK 1: Hello World – Your First Java Program

Objective

Write, compile, and run a simple Java program that prints a greeting.

Step 1.1: Create the File

1. Open **VS Code**.
2. Open your java-course folder (File → Open Folder).
3. Right-click the week1 folder in the sidebar → **New File**.
4. Name the file: HelloWorld.java
5. Press Enter.

Step 1.2: Write the Code

Copy and paste this code into HelloWorld.java:

java

// This is a single-line comment. The compiler ignores it.

```
// It is used to explain what the code does.

/*
 * This is a multi-line comment.
 * It can span multiple lines.
 * Used for longer explanations or documentation.
 */

// public: access modifier – this class is accessible from anywhere.
// class: keyword to define a class.
// HelloWorld: the name of the class (must match the file name).
public class HelloWorld {

    // public: accessible from anywhere.
    // static: this method belongs to the class, not an object.
    // void: this method does not return any value.
    // main: the entry point of any Java program.
    // String[] args: an array of strings for command-line arguments.
    public static void main(String[] args) {

        // System.out.println: prints a line of text to the console.
        // The text inside the double quotes is called a string literal.
        System.out.println("Hello, World!");

        // Print another line.
        System.out.println("My name is: [Your Name]");
```

```
        // Print another line.
        System.out.println("I am learning Java at [Your University]");

        // Print another line with the date.
        System.out.println("Today's date is: [Today's Date]");
    }
}
```

Replace the bracketed text with your own information.

Step 1.3: Run the Program

1. Press Ctrl + S to save.
2. Click the **Run** button (▶️) at the top-right of VS Code.
3. Look at the terminal at the bottom. You should see:

text

Hello, World!

My name is: John Otieno

I am learning Java at University of Nairobi

Today's date is: 2026-01-15

✓ **Task 1 Complete**

- □ I created HelloWorld.java
- □ I wrote the code with comments
- □ I ran the program successfully

TASK 2: Exploring Java Syntax Elements

Objective

Identify and classify the basic syntax elements in a Java program.

Step 2.1: Create a New File

Create a new file called SyntaxExplorer.java in the week1 folder.

Step 2.2: Write the Code

Copy and paste this code:

java

```
// This is a single-line comment
// It starts with two forward slashes

/*
 * This is a multi-line comment
 * It starts with slash-star and ends with star-slash
 */

// The class declaration
public class SyntaxExplorer {

    // The main method – entry point of the program
    public static void main(String[] args) {

        // Variable declaration and initialization
        // int is a data type for whole numbers
        // age is the variable name
        // 20 is the value assigned to age
        int age = 20;

        // double is a data type for decimal numbers
```

```
        double height = 1.75;

        // char is a data type for a single character
        char grade = 'A';

        // boolean is a data type for true/false values
        boolean isStudent = true;

        // String is a reference type for text
        String name = "Alice";

        // Print all variables with labels
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Height: " + height);
        System.out.println("Grade: " + grade);
        System.out.println("Is Student: " + isStudent);
    }
}
```

Step 2.3: Run the Program

Save and run. Expected output:

```
text
Name: Alice
Age: 20
Height: 1.75
Grade: A
```

Is Student: true

Step 2.4: Answer These Questions (Write in your notebook)

1. How many single-line comments are in the program?
2. How many multi-line comments?
3. What is the name of the class?
4. What is the name of the main method?
5. List three data types used in the program.
6. What keyword is used to print to the console?

✓ **Task 2 Complete**

- □ I created SyntaxExplorer.java
- □ I ran the program successfully
- □ I answered the questions

TASK 3: Procedural vs OOP – Rectangle Area

Objective

Compare procedural and object-oriented programming by calculating the area of a rectangle.

Step 3.1: Procedural Approach

Create a file called RectangleProcedural.java:

java

```
// Procedural approach: data and methods are separate.
public class RectangleProcedural {

    // A static method that calculates area.
    // It takes two parameters: length and width.
    // It returns the area as a double.
    public static double calculateArea(double length, double width) {
```

```
    // Multiply length and width, return the result.
    return length * width;
}

// A static method that calculates perimeter.
public static double calculatePerimeter(double length, double width) {
    // Formula: 2 * (length + width)
    return 2 * (length + width);
}

// The main method – entry point.
public static void main(String[] args) {

    // Declare variables for length and width.
    double length = 5.0;
    double width = 3.0;

    // Call the method and store the result.
    double area = calculateArea(length, width);

    // Call the perimeter method.
    double perimeter = calculatePerimeter(length, width);

    // Print the results.
    System.out.println("Procedural Approach:");
    System.out.println("Length: " + length);
```

```
        System.out.println("Width: " + width);
        System.out.println("Area: " + area);
        System.out.println("Perimeter: " + perimeter);
    }
}
```

Run it. Expected output:

text

```
Procedural Approach:
Length: 5.0
Width: 3.0
Area: 15.0
Perimeter: 16.0
```

Step 3.2: OOP Approach

Create a file called RectangleOOP.java:

java

```
// Object-oriented approach: data and methods are bundled together in a class.
public class RectangleOOP {

    // Private fields (attributes) – encapsulated.
    // They can only be accessed through methods.
    private double length;
    private double width;

    // Constructor: initializes the fields when an object is created.
    public RectangleOOP(double length, double width) {
        // 'this' refers to the current object.
```

```
    // It distinguishes the field from the parameter.
    this.length = length;
    this.width = width;
}

// Public getter for length.
public double getLength() {
    return length;
}

// Public getter for width.
public double getWidth() {
    return width;
}

// Method to calculate area.
public double getArea() {
    return length * width;
}

// Method to calculate perimeter.
public double getPerimeter() {
    return 2 * (length + width);
}

// Main method – entry point.
```

```
    public static void main(String[] args) {

        // Create an object of RectangleOOP.
        // The constructor is called with arguments 5.0 and 3.0.
        RectangleOOP rect = new RectangleOOP(5.0, 3.0);

        // Call methods on the object.
        System.out.println("OOP Approach:");
        System.out.println("Length: " + rect.getLength());
        System.out.println("Width: " + rect.getWidth());
        System.out.println("Area: " + rect.getArea());
        System.out.println("Perimeter: " + rect.getPerimeter());
    }
}
```

Run it. Expected output:

```
text
OOP Approach:
Length: 5.0
Width: 3.0
Area: 15.0
Perimeter: 16.0
```

Step 3.3: Compare the Two Approaches

Write a short paragraph (in your notebook) answering:

1. In the procedural approach, where is the data stored?
2. In the OOP approach, where is the data stored?
3. Which approach hides data better? Why?

4. Which approach is easier to extend (e.g., add color to the rectangle)? Why?

✓ **Task 3 Complete**

- □ I created RectangleProcedural.java and ran it
- □ I created RectangleOOP.java and ran it
- □ I wrote the comparison paragraph

TASK 4: Understanding JVM, JRE, JDK

Objective

Explain the roles of JVM, JRE, and JDK.

Step 4.1: Run Version Commands

Open the terminal in VS Code (Ctrl + ~) and run:

bash

java -version

Then:

bash

javac -version

Write down the output in your notebook.

Step 4.2: Write a Program to Print Java Version

Create a file called JavaInfo.java:

java

```
// This program prints information about the Java runtime.
public class JavaInfo {
    public static void main(String[] args) {
        // System.getProperty() retrieves system properties.
        // "java.version" returns the Java version.
        String version = System.getProperty("java.version");
```

```
        // "java.home" returns the installation directory of Java.
        String javaHome = System.getProperty("java.home");

        // "os.name" returns the operating system name.
        String osName = System.getProperty("os.name");

        // Print the collected information.
        System.out.println("Java Version: " + version);
        System.out.println("Java Home: " + javaHome);
        System.out.println("Operating System: " + osName);
    }
}
```

Run it. Expected output (yours may differ):

text

Java Version: 21.0.1

Java Home: C:\Program Files\Java\jdk-21

Operating System: Windows 11

Step 4.3: Answer These Questions

1. What is the difference between java and javac commands?
2. What does JVM stand for? What does it do?
3. What does JRE stand for? What does it contain?
4. What does JDK stand for? How does it relate to JRE and JVM?
5. Why can Java run on any platform?

Hint: Use Google if needed. Write answers in your notebook.

- ✓ **Task 4 Complete**

- □ I ran java -version and javac -version
- □ I created JavaInfo.java and ran it
- □ I answered the five questions

TASK 5: Push to GitHub

Objective

Commit all your Week 1 work to GitHub.

Step 5.1: Stage and Commit

1. Open the **Source Control** panel in VS Code (branch icon).
2. You should see all your new files under "Changes".
3. Click the + icon next to each file to stage them.
4. In the message box, type: Week 1: Java basics and OOP introduction
5. Click the ✓ **Commit** button.

Step 5.2: Push to GitHub

1. Click the **Sync Changes** button (or the ... menu → Push).
2. If prompted, authenticate.

Step 5.3: Verify on GitHub

1. Open your browser and go to https://github.com/YOUR-USERNAME/java-course
2. You should see your week1 folder with all the Java files.
3. Click on a file to view its contents.

✓ **Task 5 Complete**

- □ I staged all files
- □ I committed with a meaningful message
- □ I pushed to GitHub
- □ I verified my code is on GitHub

- **Final Self-Assessment Checklist**

Tick each box only when you've verified it:

- □ I created and ran HelloWorld.java
- □ I created and ran SyntaxExplorer.java
- □ I created and ran RectangleProcedural.java
- □ I created and ran RectangleOOP.java
- □ I created and ran JavaInfo.java
- □ I can explain the difference between procedural and OOP
- □ I can identify Java syntax elements (class, main, comments, statements)
- □ I can compile and run a Java program independently
- □ I can explain JVM, JRE, and JDK
- □ I pushed all my Week 1 code to GitHub

If you ticked all boxes → YOU ARE A JAVA DEVELOPER!

- **Troubleshooting Guide**

Problem	Solution
java: command not found	Restart terminal/computer. Reinstall JDK and check "Set JAVA_HOME"
VS Code won't run Java	Install "Extension Pack for Java" from Extensions marketplace
"Class names are only accepted if annotation processing is explicitly requested"	Ensure the file name matches the public class name exactly (case-sensitive)
Syntax error: ';' expected	Check that every statement ends with a semicolon

Problem	Solution
cannot find symbol	Check spelling of variable/method names, ensure they are declared
Push fails with "Authentication failed"	Use Personal Access Token instead of password
Files not showing in Source Control	Save all files (Ctrl + S), then refresh VS Code

- **Reflection Questions**

After completing this lab, answer these in your notebook:

1. What was the most challenging part of this practical session?
2. What error message did you encounter, and how did you solve it?
3. In your own words, what is the difference between procedural and OOP?
4. Why do developers use Git and GitHub?
5. How will this setup help you in your career?

- **What You've Achieved**

You have just:

- Written and run **five Java programs**
- Compared **procedural vs OOP** approaches
- Learned about **JVM, JRE, JDK**
- Pushed your code to **GitHub**

You are no longer just a student – you are a developer.

- **Next Steps**

In Week 2, you will:

1. Learn about variables, data types, and operators
2. Write programs that accept user input using Scanner
3. Build a simple calculator
4. Continue pushing your code to GitHub

Remember: Every professional developer has a GitHub account. Yours is now live!

- **Additional Resources (Self-Help)**

Topic	Resource
Java Basics	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/
OOP Concepts	https://docs.oracle.com/javase/tutorial/java/concepts/
Git Basics	https://git-scm.com/book/en/v2/Getting-Started-Git-Basics
VS Code Java	https://code.visualstudio.com/docs/java/java-tutorial

---

END OF PRACTICAL SESSION 1

"The expert in anything was once a beginner." – Helen Hayes